第二部分

算法设计—数据结构与STL

主讲人：孙歧峰

联系电话：13864717907

Q Q号码：274342509

邮　　箱：sunqf@upc.edu.cn

STL（Standard Template Library）简介

STL（Standard Template Library）是C++标准库的核心组件之一，它提供了一种通用的编程模式，使得程序员可以更加方便地实现常见的数据结构和算法。

1993年，STL正式成为C++标准库的一部分，并在1994年发布了第一个标准版本（即C++98标准）。随着C++语言的不断发展和更新，STL也在不断地完善和发展，并在C++11、C++14和C++17等标准中得到了进一步的加强和扩展。

STL的设计理念是基于泛型编程，即通过模板类和函数实现通用的数据类型和算法，使得程序员能够更加方便地实现各种数据结构和算法。STL的设计思想有三个核心要点：***容器、算法和迭代器***。

2.1.容器

容器是STL的核心组件之一，它是用于存储数据的类模板。STL提供了多种容器，包括向量（vector）、链表（list）、双端队列（deque）、栈（stack）、队列（queue）、优先队列（priority_queue）和关联容器（set、map等）。

2.2.算法

算法是STL的另一个核心组件，它是用于对容器中的元素进行操作的函数模板。STL提供了大量的算法，包括查找、排序、遍历、复制、替换、删除、去重等等。这些算法具有高度的通用性和可扩展性，可以适用于不同类型的容器和元素。

2.3.迭代器

迭代器是STL的另一个重要组成部分，它是用于遍历容器中元素的类模板。迭代器可以看做是一种指针，它可以指向容器中的元素，并支持自增、自减、解除引用等操作。

STL的诞生是C++语言的一个重要里程碑，它对C++语言和软件开发产生了深远的影响。STL的设计理念和实现方式具有高度的通用性和可扩展性，使得程序员可以更加方便地实现各种数据结构和算法。同时，STL也推动了软件开发的标准化和规范化，成为了一种通用的编程模式和设计思想。

STL的优势与不足：

？？？？？？？？？？

常用的数据结构与STL

提纲
CONTENTS

2.1 线性表

2.2 字符串

2.3 栈,队列和双端队列

2.4 二叉树和优先队列

2.5 图

2.6 哈希表

2.7 设计好的数据结构

2.1 线性表

2.1.1 线性表的定义

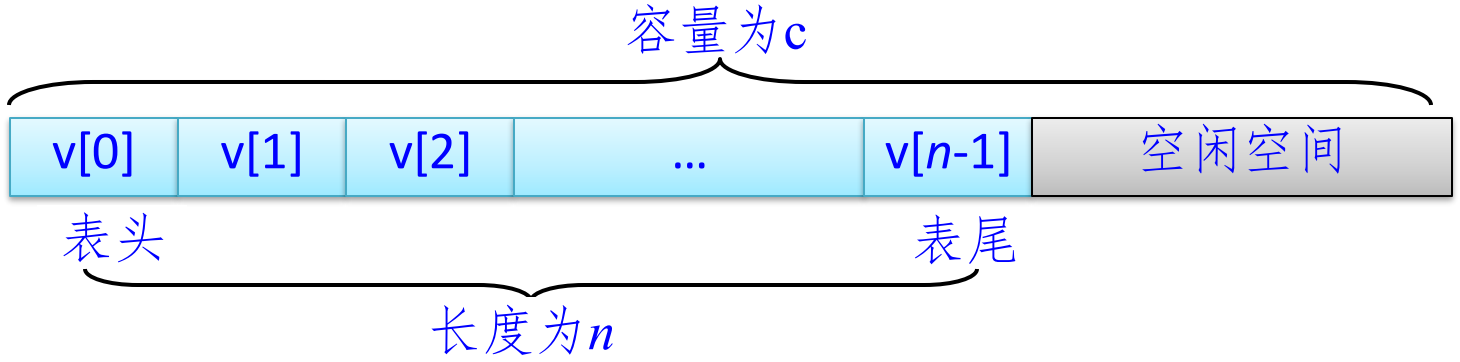
- 线性表是性质相同的n（$n \geq 0$）个元素的有限序列。
- 每个元素有唯一的序号或者位置，也称为下标或者索引，通常下标是介于0到n-1之间。
- 线性表中的n个元素从头到尾分别称为第0个元素，第1个元素，依此类推。

1. 顺序表

2. 链表

- 不带/带头结点的单链表head

2.1.2 vector向量容器



1. vector的特点

- v容器相当于动态数组，用于存储具有相同数据类型的一组元素，其容量为c（容量表示最多存放的元素个数），长度为*n*。
- v容器具有随机存取特性。
- 可以从v容器的末尾快速插入和删除元素，对应的时间复杂度为O(1)。
- 在v容器的中间插入和删除元素速度较慢。
- v容器具有自动扩容功能。

2. 定义vector向量

```
vector<int> v1;
        //定义元素为int的向量v1
vector<int> v2(2);
        //定义向量v2的容量和长度为2（2个元素均为int默认值0）
vector<double> v3(2，1.2);
        //定义向量v3的容量和长度为2（2个元素均为double值1.2）
vector<int> v4(a，a+5);
        //用数组a[0..4]共5个元素初始化向量v4
```

3. vector的成员函数

类型	成员函数	功能说明
容量	empty()	判断当前向量容器是否为空
	size()	返回当前向量容器的长度
	reserve(c)	为当前向量容器预分配c个元素的存储空间
	capacity()	返回当前向量容器的容量
	resize(n)	调整当前向量容器的长度，使其能容纳n个元素
访问元素	back()	返回当前向量容器的末尾元素
	front()	返回当前向量容器的首元素
	[idx]	返回指定下标idx的元素
	at[idx]	同[idx]

类型	成员函数	功能说明
更新	push_back(e)	在当前向量容器末尾添加了一个元素e
	emplace_back(e)	同push_back(e)，采用原地构造对象再添加
	insert(pos,e)	在pos位置插入元素e，即将元素e插入到迭代器pos指定元素之前
	emplace(pos,e)	同insert(pos,e)，采用原地构造对象再插入
	erase()	删除当前向量容器中某个迭代器或者迭代器区间指定的元素
	clear()	删除当前向量容器中所有元素
迭代器	begin()	返回当前向量容器中首元素的迭代器
	end()	返回当前向量容器中末尾元素的后一个元素的迭代器
	rbegin()	返回当前向量容器中末尾元素的迭代器
	rend()	返回当前向量容器中首元素的前一个元素的迭代器

4. vector的应用

【例2-1】假设一个整数序列采用向量容器v存放，设计一个尽可能高效的算法删除v中所有的奇数元素，要求删除后v中元素的相对次序保持不变。

解

解法1：整体建表法。先将结果v看成是一个空表，用*k*表示结果v的元素个数（初始为0），用*i*遍历v，遇到偶数时重新插入到v中，遇到奇数时跳过。最后置v的长度为*k*。

```
void delodd1(vector<int>&v)          //解法1的算法
{  int k=0;                          //k记录结果v中的元素个数
   int i=0;
   while(i<v.size())
   {  if(v[i]%2==0)                  //v[i]是偶数
      {  v[k]=v[i];                  //将v[i]重新插入到结果v中
         k++;                        //结果v的长度增1
      }
      i++;
   }
   v.resize(k);                      //设置v的长度为k
}
```

解法2：移动法。先将结果v看成是整个表，用*k*表示要删除的元素个数（初始为0），用*i*遍历v，遇到偶数时将v[*i*]前移*k*个位置，遇到奇数时将*k*增1。最后置v的长度为*n-k*。

```
void delodd2(vector<int>&v)        //解法2的算法
{   int k=0;                       //k记录删除的元素个数
    int i=0;
    while(i<v.size())
    {   if(v[i]%2==0)              //v[i]是偶数
            v[i-k]=v[i];           //将v[i]前移k个位置
        else                       //v[i]是奇数
            k++;                   //奇数元素个数增1
        i++;
    }
    v.resize(v.size()-k);          //设置v的长度为n-k
}
```

2.1.3 STL通用算法

1. 常用的通用算法

类型	函数	功能说明
非更新的序列操作	find	在指定的[beg，end)范围内查找指定值的元素
	find_if	在指定的[beg，end)范围内查找满足指定条件的元素
	count	在指定的[beg，end)范围内查找指定值出现的次数
	count_if	在指定的[beg，end)范围内查找满足指定条件的次数
更新的序列操作	copy	复制指定的[beg，end)范围内的元素
	swap	交换
	replace	将指定的[beg，end)范围内的所有值替换为新值
	replace_if	将指定的[beg，end)范围内满足条件的所有值替换为新值
	remove	删除指定的[beg，end)范围内的所有指定的值
	remove_if	删除指定的[beg，end)范围内的所有满足指定条件的值
	unique	删除相邻重复的值（并非真正删除，仅仅将保留的值前移）
	reverse	将指定的[beg，end)范围内的所有值翻转

类型	函数	功能说明
排序	***sort***	非稳定的排序
	stable_sort	稳定的排序
	is_sorted	检测有序性
有序序列的二分查找	lower_bound	在[beg，end)内查找第一个大于等于指定值的元素
	upper_bound	在[beg，end)内查找第一个大于指定值的元素
	binary_search	在指定[beg，end)范围内查找指定值的元素
合并	merge	合并两个有序序列
	set_union	求两个有序序列的并集
	set_intersection	求两个有序序列的交集
	set_difference	求两个有序序列的差集
数学	min	求两个值的最小值
	max	求两个值的最大值
	min_element	求一个序列中的最小值
	max_element	求一个序列中的最大值
	accumulate	求指定的[beg，end)范围内的值之和
其他	next_permutation	产生下一个排列
	prev_permutation	产生前一个排列

2. sort的使用方法

使用对象：像vector或者数组等具有随机存取特性的容器。

1）内置数据类型的排序

- 如果vector中元素是内置数据类型的数据，sort()默认是以less<T>（小于关系仿函数）作为比较函数实现递增排序的。
- 为了实现递减排序，需要改为greater<T>（大于关系仿函数）。

【例2-2】假设一个含n个整数的序列采用向量容器v存放，设计一个算法求第k（$1 \leqslant k \leqslant n$）大的整数（不是第$k$个不同的整数）。

解法1：将v中整数元素递增排序，那么排序后的v[n-k]就是原来v中第k大的整数。

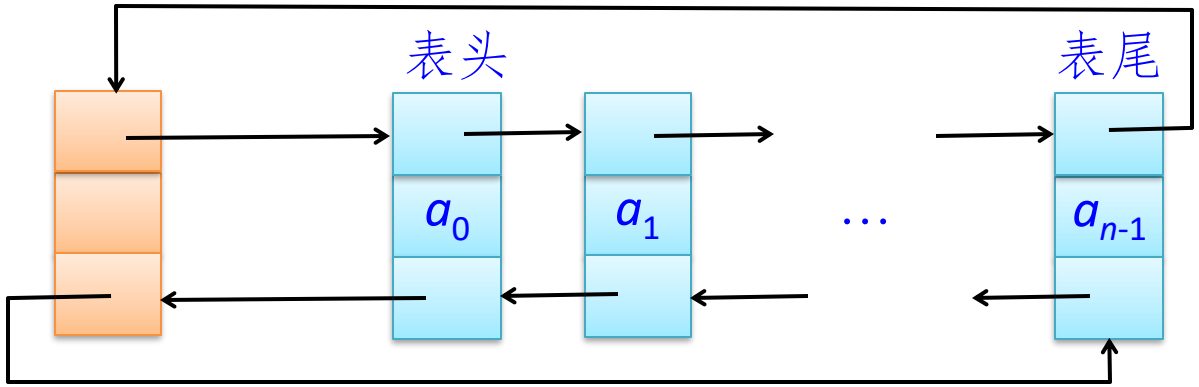
```
int topk1(vector<int> &v,int k)                //解法1的算法
{   sort(v.begin(),v.end());                   //将v中所有元素递增排序
    return v[v.size()-k];
}
```

解法2：将v中整数元素递减排序，那么排序后的v[k-1]就是原来v中第k大的整数。

```
int topk2(vector<int> &v,int k)                //解法2的算法
{  sort(v.begin(),v.end(),greater<int>());     //将v中所有元素递减排序
   return v[k-1];
}
```

2.1.4 list链表容器

1. list的特点



- ls容器采用带头结点的循环双链表实现。
- ls容器可以在任何地方快速插入与删除，对应的时间复杂度均为O(1)。
- ls容器中每个结点单独分配空间，不存在空间不够的情况。
- ls容器不具有随机存取特性，查找序号为i（$0 \leqslant i \leqslant n-1$）的元素值的时间复杂度为O($n$)。

2. 定义list容器

```
list<int> l1;
                    //定义元素为int的链表l1
list<int> l2 (10);
                    //指定链表l2的初始大小为10个int元素
list<double> l3 (10，1.23);
                    //指定l3的10个初始元素的初值为1.23
list<int> l4(a，a+5);
                    //用数组a[0..4]共5个元素初始化l4
```

3. list的成员函数

类型	成员函数	功能说明
容量	empty()	判断链表容器是否为空
	size()	返回链表容器的长度
访问元素	back()	返回链表容器的末尾元素
	front()	返回链表容器的首元素
更新	push_back(e)	在链表尾部插入元素e
	emplace_back(e)	同push_back(e)，采用原地构造对象再添加
	push_front(e)	在链表首部插入元素e
	emplace_front(e)	同push_front(e)，采用原地构造对象再添加
	insert(pos,e)	在pos位置插入元素e
	insert(pos,n,e)	在pos位置插入n个元素e
	insert(pos,pos1,pos2)	在迭代器pos处插入[pos1,pos2)的元素
	pop_back()	删除链表容器的末尾元素
	pop_front()	删除链表容器的首元素。
	erase()	从链表容器中删除一个或几个元素
	clear()	删除链表容器中所有的元素

类型	成员函数	功能说明
操作	remove()	删除链表容器中所有指定值的元素
	remove_if(cmp)	删除链表容器中满足条件的元素
	unique()	删除链表容器中相邻的重复元素
	merge()	合并两个有序链表容器中的元素
	reverse()	反转当前链表容器的所有元素
	sort()	对链表容器中的元素排序
迭代器	begin()	返回当前链表容器中首元素的迭代器
	end()	返回当前链表容器中末尾元素的后一个元素的迭代器
	rbegin()	返回当前链表容器中末尾元素的迭代器
	rend()	返回当前链表容器中首元素的前一个元素的迭代器



STL提供的sort()排序算法仅支持具有随机存取特性的容器，而list容器不支持随机访问，为此，list链表容器提供了sort()成员函数用于元素排序，类似的还有unique()、reverse()、merge()等成员函数。

2.2 字符串

2.2.1 字符串的定义

- 字符串简称为串，是字符的有限序列，可以看成元素类型是字符的线性表。
- 一个串s中若干连续的字符构成的串t称为s的子串。
- 空串是任何串的子串。
- 两个串相等当且仅当它们的长度相同并且对应位置的字符均相同。

1. 顺序串

```
struct SqString                 //顺序串类型
{   char data[MaxSize];         //MaxSize为data数组的容量
    int length;                 //顺序串长度即实际元素个数

    SqList():length(0) { }      //构造函数
};
```

2. 链串

```
struct SNode                                        //链串的结点类型
{   char val;                                       //结点值
    ListNode *next;                                 //后继结点的地址

    ListNode() : val(0), next(NULL) {}              //默认构造函数
    ListNode(char x) : val(x), next(NULL) {}        //重载构造函数1
    ListNode(char x, SNode *next):val(x), next(next) {}   //重载构造函数2
};
```

2.2.2 string字符串容器

1. 定义string容器

- string ()：建立一个空的字符串。
- string (const string& str)：用字符串str建立当前字符串。
- string(const string& str, size_type idx)：用字符串str起始于idx的字符建立当前字符串。
- string (const string& str, size_type idx, size_type num)：用字符串str起始于idx的num个字符建立当前字符串。
- string(const char* cstr)：用C-字符串cstr建立当前字符串。
- string (const char* chars, size_type num)：用C-字符串cstr开头的num个字符建立当前字符串。
- string (size_type num, char c)：用num个字符c建立当前字符串。

2. string的成员函数

类型	成员函数	功能说明
容量	empty()	判断当前字符串是否为空串
	size()	返回当前字符串的长度
	length()	与size()相同
访问元素	back()	返回当前字符串容器的末尾元素
	front()	返回当前字符串容器的首元素
	[idx]	返回当前字符串位于idx位置的字符，idx从0开始
	at(idx)	返回当前字符串位于idx位置的字符

类型	成员函数	功能说明
更新	append(str)	在当前字符串的末尾添加一个字符串str
	push_back(c)	在当前字符串的末尾添加一个字符c
	insert(size_type idx,const string&str)	在当前字符串的idx序号处插入一个字符串str
	replace(size_type idx,size_type num,const string&str)	将当前字符串中起始于idx的num个字符用一个字符串str替换
	replace(iterator beg,iterator end,const string&str)	将[beg,end)区间的所有字符用字符串str替换
	pop_back()	删除当前字符串的末尾字符
	erase()	删除当前字符串中的所有的字符
	erase(size_type idx)	删除当前字符串的从idx开始的所有字符
	erase(size_type idx,size_type num)	删除当前字符串的从idx开始的num个字符
	clear()	删除当前字符串中的所有的字符

类型	成员函数	功能说明
字符串操作	find(string& s, size_type pos=0)	在当前字符串中从pos位置（默认为0）开始向后查找字符串s的第一个位置
	rfind(string&s,size_type pos=npos)	在当前字符串中从pos位置（默认为npos）开始向前查找字符串s的第一个位置
	substr(size_type idx)	返回当前字符串起始于idx的子串
	substr(size_type idx,size_type num)	返回当前字符串起始于idx的长度为num的子串
	compare(const string&str)	返回当前字符串与字符串str的比较结果。在比较时，若两者相等则返回0，前者小于后者则返回-1，否则返回1
迭代器	begin()	返回当前字符串容器中首元素的迭代器
	end()	返回当前字符串容器中末尾元素的后一个元素的迭代器
	rbegin()	返回当前字符串容器中末尾元素的迭代器
	rend()	返回当前字符串容器中首元素的前一个元素的迭代器

3. string的应用

【例2-4】假设两个字符串s和t均采用string容器存储。设计一个算法利用string的成员函数求t在s中不重叠出现的次数。例如s="aaaaa"，t="aa"，结果为2。

解

```
int Count(string&s,string&t)
{   int cnt=0;
    int pos=s.find(t,0);
    while(pos!=string::npos)        //在s中找到t
    {   cnt++;
        pos=s.find(t, pos+t.size());        //继续在后续字符中查找
    }
    return cnt;
}
```

npos是一个常数，表示size_t的最大值（Maximum value for size_t）。许多容器都提供这个，用来表示不存在的位置。在使用string类的find()、rfind()、find_first_of()、find_last_of()、find_first_not_of()、substr()、erase()和find_last_not_of()等函数时，如果这些函数未能找到匹配的字符或子串，则会返回string::npos。这意味着，当这些函数的返回值等于string::npos时，表示在字

【例2-5】假设有3个字符串str、s和t，均采用string容器存储。设计一个算法利用string的成员函数将str中所有的子串s用t串替换。例如str="ababcd"，s="ab"，t="123"，替换后str变为"123123cd"。

解

```
void Replaceall(string&str,string&s,string&t)
{   int m=s.size();
    int pos=str.find(s,0);
    while(pos!=string::npos)          //在str中找到s
    {   str.replace(pos,m,t);         //替换
        pos=str.find(s,pos+t.size());          //继续在后续字符中查找
    }
}
```

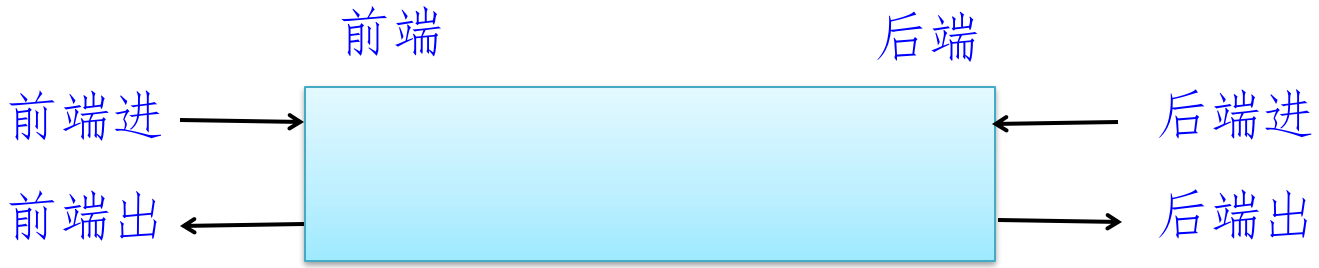
2.3 栈、队列和双端队列

2.3.1 栈、队列和双端队列的定义

- 栈是一种特殊的线性表，有前、后两个端点，规定只能在其中一端进行进栈和出栈操作。
- 队列是一种特殊的线性表，有前、后两个端点，规定只能在一端进队元素，另外一端出队元素。
- 双端队列是一种特殊的线性表，有前、后两个端点，每个端点都可以进队和出队元素。

2.3.2 deque双端队列容器

1. deque的特点



- dq容器在实现上由若干个块构成，每个块中元素地址是连续的，块之间的地址是不连续的，系统有一个特定的机制将这些块构成一个整体并且维护相关操作。
- 由于dq容器采用分块存放容器中的元素，所以空间的重新分配要比vector快，因为重新分配空间后，原有的元素不需要复制。
- dq容器提供了随机访问迭代器，但随机访问的时间不再是O(1)，也远好于O(*n*)。
- dq容器在前后端进队和出队除元素的时间复杂度均为O(1)，但在中间位置插入和删除元素速度较慢。
- dq容器不同于队列，它提供了遍历元素的迭代器成员函数，可以正向或者反向遍历其中的全部元素。

2. 定义deque容器

```
deque<int> dq1;
                    //定义元素类型为int的空双端队列dq1

deque<int> dq2(10);
                    //指定dq2初始为10个int元素

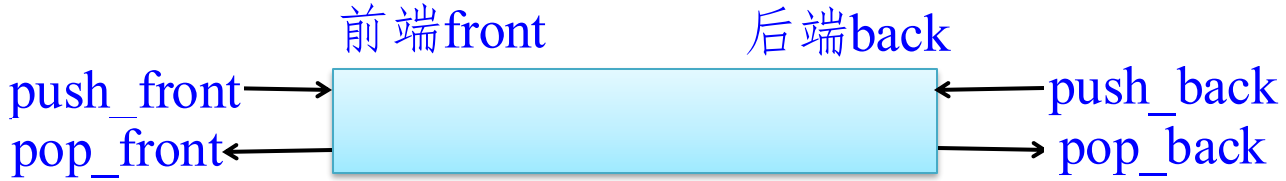
deque<double> dq3(10,1.23);
                    //指定dq3的10个初始元素均为1.23

deque<int> dq4(dq2.begin(),dq2.end());
                    //用dq2的所有元素初始化dq4
```

3. deque的成员函数

类型	成员函数	功能说明
容量	empty()	判断双端队列容器是否为空队
	size()	返回双端队列的长度
访问元素	back()	返回当前双端队列容器的末尾元素
	front()	返回当前双端队列容器的首元素
	[idx]	返回指定下标idx的元素
	at[idx]	同[idx]

类型	成员函数	功能说明
容量	empty()	判断双端队列容器是否为空队
更新	push_front(e)	在队头插入元素e
	push_back(e)	在队尾插入元素e
	pop_front()	出队一个队头元素
	pop_back()	出队一个队尾元素
	insert()	在双端队列容器中插入一个或几个元素
	erase()	从双端队列容器中删除一个或几个元素
	clear()	出队双端队列容器中所有元素
迭代器	begin()	返回当前双端队列容器中首元素的迭代器
	end()	返回当前双端队列容器中末尾元素的后一个元素的迭代器
	rbegin()	返回当前双端队列容器中末尾元素的迭代器
	rend()	返回当前双端队列容器中首元素的前一个元素的迭代器



说明

deque容器在出队元素函数（pop_front、pop_back）中并不检测队空，所以在调用这些函数时务必保证容器是非空的，否则会导致程序停止正常工作。

4. deque的应用

【例2-6】给定一个含n个整数的序列 nums 和滑动窗口的大小 k（$1 \leqslant k \leqslant n$），设计一个算法求出所有滑动窗口里的最大值。

例如，nums=（4，3，5，4，3，3，6，7），k=3。

nums: [4, 3, 5], 4, 3, 3, 6, 7 ➡ 5
nums: 4, [3, 5, 4], 3, 3, 6, 7 ➡ 5
nums: 4, 3, [5, 4, 3], 3, 6, 7 ➡ 5
nums: 4, 3, 5, [4, 3, 3], 6, 7 ➡ 4
nums: 4, 3, 5, 4, [3, 3, 6], 7 ➡ 6
nums: 4, 3, 5, 4, 3, [3, 6, 7] ➡ 7

结果是
（5，5，5，4，6，7）

N-k+1个元素

普通算法：效率多少？

更高效的算法?

高效算法：



单调队列处理滑动数组问题

- 前端大，保持前端元素为当前滑动窗口的最大元素。
- 处理nums[i]:将队尾小于nums[i]的所有元素从队尾元素出队，再将nums[i]从队尾进队。
- 队头过期的元素从队头出队，为此nums[i]进队时改为求*序号i*进队。



解 高效算法： 维护一个递减的序列

```
vector<int> maxSlidingWindow(vector<int>&nums,int k)
{    int n=nums.size();
     deque<int> dq;//用队头元素表示当前滑动窗口内最大值的位置
     vector<int> ans;//记录滑动窗口元素的最大值

     //处理nums前k个元素

     for(int i=0;i<k;i++)
     {    if(dq.empty()) //队空时将元素下标i进队尾
               dq.push_back(i);
          else //队不空时
          {    while(!dq.empty() && nums[i]>nums[dq.back()])
                    dq.pop_back();//将队尾小于nums[i]的元素从队尾出队
               dq.push_back(i); //将元素下标i进队尾
          }
      }
     ans.push_back(nums[dq.front()]);//队头元素添加到ans中
     //处理其余的n-k个元素
      for(int i=k;i<n;i++)        //处理nums中剩余的元素
      {    if(dq.empty()) //队空时将元素下标i进队尾
                dq.push_back(i);
           else //队不空时
           {    while(!dq.empty() && nums[i]>nums[dq.back()])
                     dq.pop_back();  //将队尾小于nums[i]的元素从队尾出队
                dq.push_back(i);//将元素下标i进队尾
           }

           if(dq.front()<i-k+1)  //将队头过期的元素从队头出队
           dq.pop_front();

           ans.push_back(nums[dq.front()]); //新队头元素添加到ans中
      }
     return ans;
}
```

维护递减序列的同时，处理过期元素

从头到尾遍历一遍完成，高效！

2.3.3 queue队列容器

1. queue的特点

- qu容器只能在队头出队（删除）、在队尾进队（插入）元素，不能在中间位置删除和插入元素，因此队中元素先进先出。
- qu容器具有自动扩容功能，不必考虑队满的情况。
- qu容器中的元素不允许顺序遍历，不支持begin()/end()和rbegin()/rend()迭代器函数。

2. 定义queue容器

```
queue<int> qu1;              //定义int类型的空队列qu1（底层容器是deque）
list<int> ls(3,2);           //定义含3个元素值均为2的ls容器
queue<int,list<int>> qu2(ls);    //由ls创建队列qu2
```

3. queue的成员函数

成员函数	功能说明
empty()	判断队列容器是否为空
size()	返回队列容器的长度
front()	返回队头元素
back()	返回队尾元素
push(e)	进队元素e
pop()	出队一个元素



queue容器在出队元素函数（pop）中并不检测队空，所以在调用该函数时务必保证容器是非空的，否则会导致程序停止正常工作。

4. queue的应用

【例2-7】给定一个含n（n>1）个整数的队列容器qu，设计一个算法出队其中序号为k（1≤k≤n）的元素（从队头开始数，队头元素的序号为1），并返回该元素。

解




```
int popk(queue<int>&qu,int k)
{   int n=qu.size(),x;
    for(int i=1;i<k;i++)                //处理序号1到k-1的元素
    {   x=qu.front(); qu.pop();                 //出队元素x
        qu.push(x);                     //将x进队
    }
    int e=qu.front(); qu.pop();         //出队序号为k的元素e
    for(int i=k+1;i<=n;i++)             //处理序号k+1到n的元素
    {   x=qu.front(); qu.pop();         //出队元素x
        qu.push(x);                     //将x进队
    }
    return e;
}
```

O(n)

2.3.4 stack栈容器

1. stack的特点

- st容器只能在栈顶出栈（删除）和进栈（插入）元素，不能在中间位置删除和插入元素，因此栈中元素后进先出。
- st容器具有自动扩容功能，不必考虑栈满的情况。
- st容器中的元素不允许顺序遍历，不支持begin()/end()和rbegin()/rend()迭代器函数。

2. 定义stack容器

```
stack<int> st1;  //定义int类型的空栈st1（底层容器是deque）
vector<int> v(2,10);       //定义含2个元素值均为10的v容器
stack<int,vector<int>> st2(v);                  //由v容器创建st2
```

3. stack的成员函数

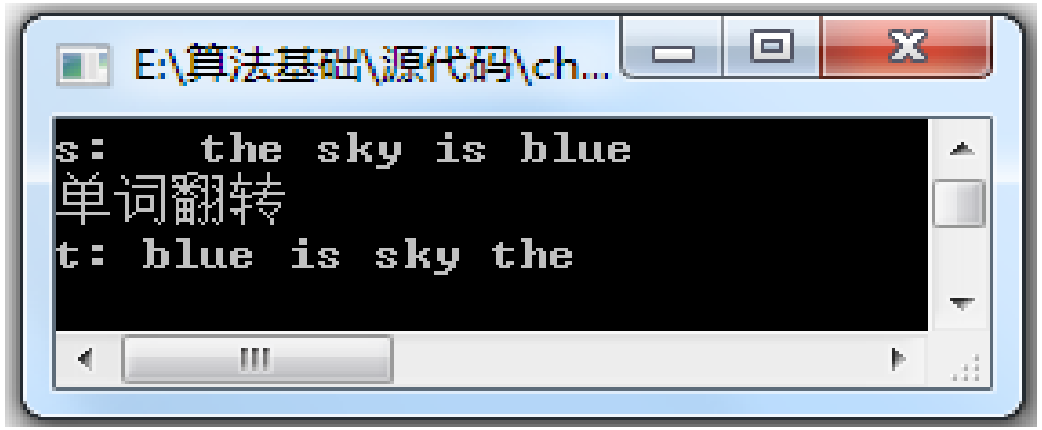
成员函数	功能说明
empty()	判断栈容器是否为空
size()	返回栈容器的长度
push(e)	进栈元素e
top()	返回栈顶元素
pop()	元素一个出栈



stack容器在出栈元素函数（pop）中并不检测栈空，所以在调用该函数时务必保证容器是非空的，否则会导致程序停止正常工作。

【例2-8】给定一个含若干单词的字符串s，单词之间用一个或者多个空格分隔。设计一个算法将s中所有单词翻转并且返回翻转后的结果，结果字符串中两个单词之间只有一个空格。

例如s="the sky is blue"翻转的结果t="blue is sky the"。



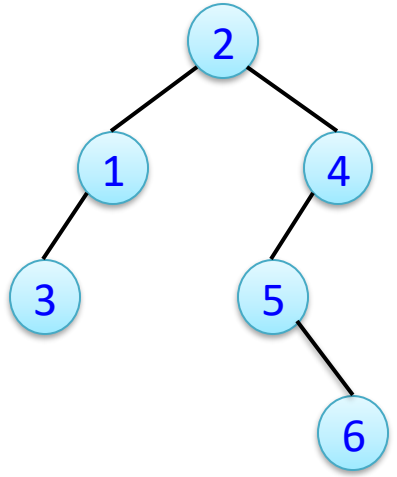
解

```
string Reversewords(string& s)
{   stack<string> st;
    int i=0;
    while(i<s.length())                                  //遍历字符串s
    {   while(i<s.length() && s[i]==' ')   i++;          //跳过空字符
        string tmp="";
        while(i<s.length() && s[i]!=' ')                 //提取单词tmp
        {   tmp+=s[i];  i++;  }
        st.push(tmp);                                    //将tmp进栈
    }

    string ans="";
    while(!st.empty())                          //出栈所有单词连接成字符串ans
    {   if(ans.length()==0)
            ans+=st.top();
        else
            ans+=" "+st.top();
        st.pop();
    }
    return ans;
}
```

这里用 Stringstream + stack 更简单

2.4 二叉树和优先队列



1. 二叉树的定义

- 二叉树是有限的结点集合。
- 这个集合或者是空，或者由一个根结点和两棵互不相交的称为左子树和右子树的二叉树组成。

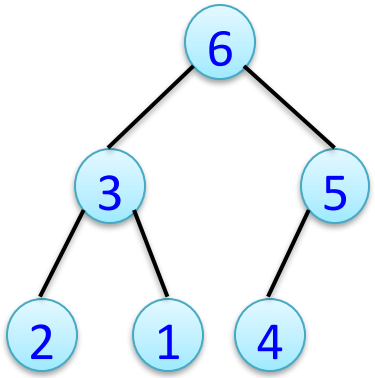
2. 优先队列的定义

- 优先队列中元素被赋予优先级。
- 出队的元素总是当前具有最高优先级的元素。

2.4 优先队列

3. 堆的定义

(6,3,5,2,1,4)



每个结点值不小于其孩子结点值

大根堆

4. 优先队列

大根堆

- 用*R*[0..*n*-1]存储。底层容器可以用Vector或queue，但不能用list。
- 堆的主要运算是筛选，分为自顶向下和自底向上两种。

2.4.2 priority_queue优先队列容器

1. priority_queue的特点

- pq容器是用数组实现的，但是数组容量可以动态增加，容量无限。
- 定义pq容器时需要指定元素优先级的比较函数，默认为less<T>。
- pq容器中的元素不允许顺序遍历，不支持begin()/end()和rbegin()/rend()迭代器函数。

2. 定义priority_queue容器

1）元素为内置数据类型的堆

- 默认是less<T>（小于关系函数）作为关系函数，元素值越大优先级的越高（即默认创建大根堆）.
- 可以改为以greater<T>作为关系函数，这样元素值越小优先级的越高（即创建小根堆）。

2）元素为自定义类型的堆

- 在结构体类型中重载<运算符，以指定优先级，如priority_queue<Stud> pq1调用默认的<运算符创建堆pq1（是大根堆还是小根堆由<重载函数体确定）。
- 在结构体类型中重载>运算符，以指定优先级，如priority_queue<Stud, vector<Stud>，greater<Stud>> pq2调用重载>运算符创建堆pq2，此时需要指定优先队列的低层容器（这里为vector，也可以是deque）。
- 自定义包含关系比较函数()的结构体Cmp，在关系比较函数()中指定优先级方式，如priority_queue<Stud,vector<Stud>,Cmp> pq3调用Cmp的()运算符创建堆pq3，此时需要指定优先队列的低层容器（这里为vector，也可以是deque）。

3. priority_queue的成员函数

成员函数	功能说明
empty()	判断优先队列容器是否为空
size()	返回优先队列容器的长度
top()	返回队头元素
push(e)	进队元素e
pop()	出队一个元素



priority_queue容器在出队元素函数（pop）中并不检测队空，所以在调用该函数时务必保证容器是非空的，否则会导致程序停止正常工作。

4. priority_queue的应用

【例2-9】给定一个含n（n>1）个整数的序列，设计一个算法求前k（1≤k≤n）个最小的整数，结果按递增顺序排列。

解法1： 用小根堆优先队列pq求解。

```
vector<int> topk1(vector<int>&v,int k)
 {  vector<int> ans;
    priority_queue<int,vector<int>,greater<int>> pq(v.begin(),v.end());
    for(int i=1;i<=k;i++)
    {   ans.push_back(pq.top());
        pq.pop();
    }
    return ans;
}
```

解法2：用大根堆优先队列pq求解。

```
vector<int> topk2(vector<int>&v,int k)
{   vector<int> ans(k);                  //指定ans的长度为k
    priority_queue<int> pq;              //定义大根堆pq
    for(int i=0;i<k;i++)                 //将前k个整数进队
        pq.push(v[i]);
    for(int i=k;i<v.size();i++)          //处理v中剩余的元素
    {   if(v[i]<pq.top())
        {   pq.pop();
            pq.push(v[i]);
        }                                        用大根堆维护最小的K个元素
    }
    int j=k-1;
    while(!pq.empty())       //出队pq中全部整数并插入到ans前面
    {   ans[j--]=pq.top();
        pq.pop();
    }
    return ans;
}
```

算法效率和优势？？

2.7.3 集合容器set/multiset

1. 集合容器的特点

以set为例

- s容器采用红黑树（自平衡二叉树）存储的，所以是按关键字有序的，默认情况下所有元素按关键字递增排列。按关键字查找时间为$O(\log_2 n)$。
- s容器通过关键字key来存储和读取元素，这些关键字与元素在容器中的位置无关，所以不提供顺序容器中的push_front()、push_back()、pop_front()和pop_back()操作。
- 增加元素通过insert()实现，删除元素通过erase()实现。
- 由于s容器中元素的关键字是唯一的，向其中插入元素时，如果已经存在则不插入。

定义set容器

```
set<int> s1; //创建一个int类型的空集合s1（排列方式是递增）
set<int,greater<int>> s2;    //创建一个int类型的空集合s1（排列方式是递减）
set<int> s3(s1);    //创建一个与s1相同的集合s3
```

3. set的成员函数

类型	成员函数	功能说明
容量	empty()	判断容器是否为空
	size()	返回容器的长度
更新	insert(e)	插入元素e
	erase()	从容器删除一个或几个元素
	clear()	删除所有元素
操作	count(k)	返回容器中关键字k出现的次数，结果为0或者1
	find(k)	如果容器中存在关键字为k的元素，返回该元素的迭代器，否则返回set::end()值
	lower_bound(k)	返回第一个关键字大于等于k的元素的迭代器
	upper_bound(k)	返回第一个关键字大于k的元素的迭代器
迭代器	begin()	返回当前容器中首元素的迭代器
	end()	返回当前容器中末尾元素的后一个元素的迭代器
	rbegin()	返回当前容器中末尾元素的迭代器
	rend()	返回当前容器中首元素的前一个元素的迭代器

4. set的应用

【例2-11】给定一个无序的整数序列v，设计一个算法求第k（1≤k≤v中不同的元素个数）大的整数（是指第k个不同值的元素），例如，v=（1，1，2，2，3，3，4，4，5），k=4，结果为2。

解

```
int topk(vector<int>&v,int k)
{   set<int,greater<int>> s;
    for(auto e:v)    //遍历V中的元素
        s.insert(e);
    auto it=s.begin();                    //定义s的迭代器
    for(int i=1;i<k;i++)                  //it向后移动k-1次
        it++;
    return *it;
}
```

2.7.4 映射容器map/multimap

1. 映射容器的特点

以map为例

- mp容器采用红黑树存储的，所以是按关键字有序的，默认情况下所有元素按关键字递增排列，按关键字查找的时间复杂度为$O(\log_2 n)$。
- mp容器通过关键字key来存储和读取元素，这些关键字与元素在容器中的位置无关，所以不提供顺序容器中的push_front()、push_back()、pop_front()和pop_back()操作。增加元素通过insert()实现，删除元素通过erase()实现。
- 由于mp容器中元素的关键字key是唯一的，向其中插入元素时，如果已经存在则不插入。

2. 定义map容器

```
map<string,int> mp1;
                    //创建一个空映射mp1（排列方式是递增）
map<int,int,greater<int>> mp2;
                    //创建一个空映射mp2（排列方式是递减）
map<int,int,greater<int>> mp3(mp2);
                    //创建一个与mp2相同的映射mp3
```

3. map的成员函数

类型	成员函数	功能说明
容量	empty()	判断容器是否为空
	size()	返回容器中实际元素个数
访问元素	map[k]	返回关键字为k的元素的引用，如果不存在这样的关键字，则以k作为关键字插入一个元素（不适合multimap）
	at[k]	同map[k]
更新	insert(e)	插入一个元素e并返回该元素的位置
	erase(k)	从容器删除元素k
	erase(it)	从容器删除迭代器it指向的元素
	erase(beg,end)	从容器删除[beg,end)迭代器范围的元素
	clear()	删除所有元素
类型	成员函数	功能说明
操作	find(k)	在容器中查找关键字为k的元素
	count(k)	返回容器中关键字为k的元素个数，结果为0或者1
	lower_bound(k)	返回第一个关键字大于等于k的元素的迭代器
	upper_bound(k)	返回第一个关键字大于k的元素的迭代器
迭代器	begin()	返回当前容器中首元素的迭代器
	end()	返回当前容器中末尾元素的后一个元素的迭代器
	rbegin()	返回当前容器中末尾元素的迭代器
	rend()	返回当前容器中首元素的前一个元素的迭代器

4. map的应用

【例2-12】给定一个无序的整数序列v，设计一个算法求其中最大的k（$1 \leqslant k \leqslant$v中不同的元素个数）个整数（是指最大的k个不同值的整数）出现的次数，例如，v=（1，1，2，3，3），k=3，结果为（3，2，2，1，1，2），表示3出现2次，2出现1次，1出现2次。

解

```
vector<int> topck(vector<int>&v,int k)
{   vector<int> ans;                    //存放结果
    map<int,int,greater<int>> mp;
    for(auto e:v)
        mp[e]++;
    auto it=mp.begin();                 //定义mp的迭代器
    for(int i=1;i<=k;i++)               //求前k个最大整数及其出现次数
    {   ans.push_back(it->first);
        ans.push_back(it->second);
        it++;
    }
    return ans;
}
```

Set与map的不同：

1：存储的数据类型不同：

- Set：Set 是一种存储唯一值的集合，它的成员值只能出现一次，不重复。可以存储任何类型的值，包括原始值和引用值。
- Map：Map 是一种键值对的集合，它将键与值相关联。键是唯一的，每个键只能出现一次，但值可以重复。可以使用各种类型的值作为键和值。

2：存储方式的差异：

- Set：Set 使用类似于数组的方式存储值，保持插入顺序。它通过值的相等性（通过 "SameValueZero" 算法判断）来确定唯一性。
- Map：Map 使用键值对的方式存储数据，可以通过自定义的键来关联和访问值。键和值的插入顺序被保持。

3：迭代方式的不同：

- Set：Set 提供了迭代器（Iterator）接口，可以使用 for...of 循环或 forEach 方法对 Set 进行迭代。
- Map：Map 提供了迭代器接口，可以使用 for...of 循环或 forEach 方法对 Map 的键值对进行迭代。

4：大小和性能的差异：

- Set：Set 的大小是根据其包含的唯一值的数量来计算的。对于大型数据集合，Set 的查询性能较好。
- Map：Map 的大小是根据其键值对的数量来计算的。对于需要频繁增删键值对的操作，Map 通常具有更好的性能。

5：使用场景的不同：

- Set：Set 适用于需要存储一组唯一值，并且不关心顺序的场景。例如，去重、判断元素是否存在等。
- Map：Map 适用于需要将值与特定键关联的场景。例如，存储键值对的配置信息、构建字典、缓存等。

2.8.2 哈希集合容器unordered_set

1. unordered_set的特点

- s容器采用哈希表实现，利用拉链法解决冲突，所有元素是无序排列的，按关键字查找的时间复杂度接近O(1)。
- s容器是一个关联容器，其中的元素只能根据关键字来访问，不能按索引访问。
- s容器中所有元素的关键字是唯一的。

去重无序

2. 定义unordered_set容器

```
unordered_set<int> s1;                    //创建一个int类型的空哈希集合s1
unordered_set<int> s2(s1);                //创建一个与s1相同的哈希集合s2
```

3. unordered_set的成员函数

类型	成员函数	功能说明
容量	empty()	判断容器是否为空
	size()	返回容器的长度
更新	insert(e)	插入元素e
	erase()	从容器删除一个或几个元素
	clear()	删除所有元素
元素查找	count(k)	返回容器中关键字k出现的次数，结果为0或者1
	find(k)	如果容器中存在关键字为k的元素，返回该元素的迭代器，否则返回set::end()值
迭代器	begin()	返回当前容器中首元素的迭代器
	end()	返回当前容器中末尾元素的后一个元素的迭代器

4. unordered_set的应用

【例2-13】给定一个无序的整数序列v，设计一个算法求其中不相同的元素个数。

去重

解

```
int Count(vector<int>&v)
{    unordered_set<int> s;
     for(auto e:v)
          s.insert(e);
     return s.size();
}
```

2.8.3 哈希映射容器unordered_map

1. unordered_map的特点

- mp容器采用哈希表实现，利用拉链法解决冲突，所有元素是无序排列的，按关键字查找的时间复杂度接近O(1)。
- mp容器是一个关联容器，其中的元素只能根据关键字来访问，不能按索引访问。
- mp容器中所有元素的关键字是唯一的。

2. 定义unordered_map容器

```
unordered_map<string,int> mp1;                  //创建<string,int>类型的空哈希映射mp1
unordered_map<string,int> mp2(mp1);             //创建一个与mp1相同的哈希映射mp2
unordered_map<Stud,int,MyHash,MyEqualTo> mp3;   //创建mp3，指定哈希函数和=
```

3. unordered_map的成员函数

类型	成员函数	功能说明
容量	empty()	判断容器是否为空
	size()	返回容器中实际元素个数
元素访问	map[k]	返回关键字为k的元素的引用，如果不存在这样的关键字，则以k作为关键字插入一个元素（不适合multimap）
	at[k]	同ma[k]
更新	insert(e)	插入一个元素e并返回该元素的位置
	erase(k)	从容器删除元素k
	erase(k),erase(it)	从容器删除关键字为k的元素或者迭代器it指向的元素
	erase(beg,end)	从容器删除[beg,end)迭代器范围的元素
	clear()	删除所有元素
元素查找	find(k)	在容器中查找关键字为k的元素
	count(k)	返回容器中关键字为k的元素个数，结果只有1或者0
迭代器	begin()	返回当前容器中首元素的迭代器
	end()	返回当前容器中末尾元素的后一个元素的迭代器

4. unordered_map的应用

【例2-14】设计一个算法判断字符串str中每个字符是否唯一。如"abc"的每个字符是唯一的，算法返回true，而"accb"的中字符'c'不是唯一的，算法返回false。

解

```
bool isUnique(string &str)
{   unordered_map<char,int> mp;
    for (int i=0;i<str.length();i++)
    {   mp[str[i]]++;
        if (mp[str[i]]>1)
            return false;
    }
    return true;
}
```

2.9 设计好的数据结构

-----高效算法的基础

【例2-15】某个系统的操作如下：

1 *x*：输入整数*x*，所有输入的整数不会重复。

2：删除前面输入序列中的最大整数并输出。

3：删除前面输入序列中的最小整数并输出。

4：按递增顺序输出当前的整数序列。

选择适合的数据结构并尽可能高效地设计上述算法。操作序列用vector<int>容器v表示，假设操作序列是正确的并且执行操作2和3时，当前的输入序列至少有一个整数。

例如，v=（1，2，1，5，2，1，6，3，4），表示输入整数2和5，执行一次2操作，输入6，再执行一次3操作，最后执行一次4操作。

解 从操作2～4看出输入的整数序列应该有序存储，又由于输入的整数不会重复，为此采用set容器存放（默认递增排列），set容器是一种平衡二叉树，这样操作1～3对应时间复杂度均为$O(\log_2 n)$。

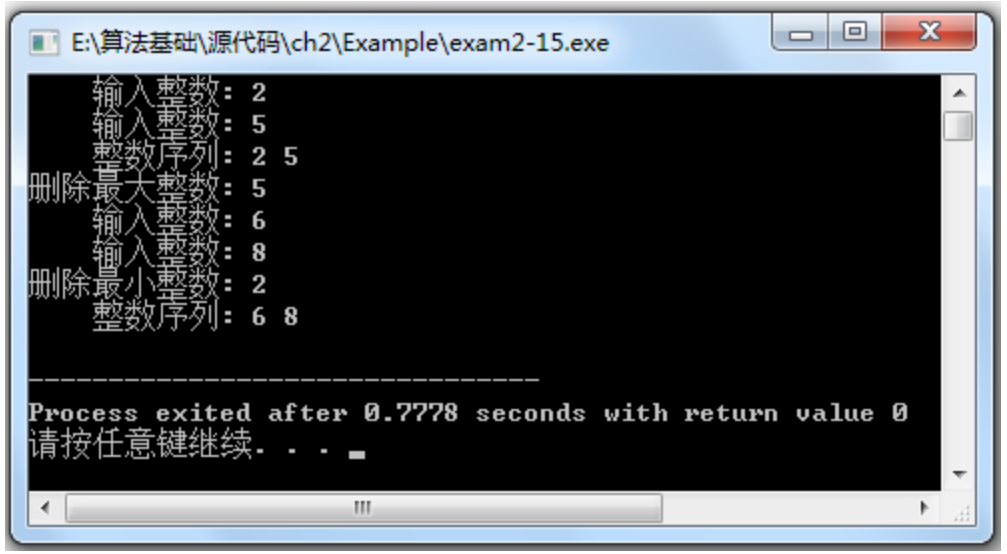
```
set<int> s;                          //定义集合容器s

void solve(vector<int>& v)           //求解算法
{   int i=0,x;
    while(i<v.size())
    {   if(v[i]==1)                  //操作1
        {   i++;
            x=v[i];  s.insert(x);
            printf("   输入整数: %d\n",x);
        }
        else if(v[i]==2)             //操作2
        {   auto it=s.end();
            it--;
            printf("删除最大整数: %d\n",*it);
            s.erase(it);
        }
```

```
        else if(v[i]==3)                    //操作3
        {   auto it=s.begin();
            printf("删除最小整数: %d\n",*it);
            s.erase(it);
        }
        else                                //操作4
        {   printf("   整数序列: ");
            for(auto it=s.begin();it!=s.end();it++)
                printf("%d ",*it);
            printf("\n");
        }
        i++;
    }
}
```

程序验证

```
int main()
{   vector<int> v={1,2,1,5,4,2,1,6,1,8,3,4};
    solve(v);
    return 0;
}
```



【例2-16】某个系统的操作如下：

1 *x*：输入整数*x*，输入的整数可能重复。

2：求前面输入序列的中位数并输出，中位数是***有序序列***中间的数，如果序列长度是偶数，中位数则是中间两个数的平均值。

对顶堆

选择适合的数据结构并尽可能高效地设计上述算法。操作序列用vector<int>容器v表示，假设操作序列是正确的并且执行操作2时当前的输入序列至少有一个整数。

例如，v=（1，2，1，6，1，3，1，2，2，1，5，2），表示输入整数2，6，3，2，求一次中位数（结果是2.5），输入5，再求一次中位数（结果是3）。

解

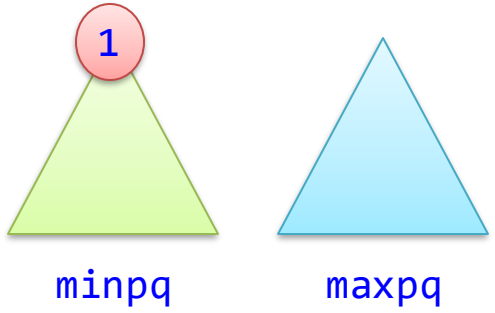
- 用两个堆,即小根堆minpq和大根堆maxpq来实现。
- minpq存放最大的一半整数（根最小），maxpq存放最小的一半整数（根最大）。
- 当两个堆中共有偶数个整数时，保证两个堆中整数个数相同。
- 当两个堆中共有奇数个整数时，保证小根堆中多一个整数。

对于输入的整数*x*，操作过程如下：

- 若小根堆minpq为空，将*x*插入到minpq中，然后返回。
- 若x大于小根堆堆顶元素，将*x*插入到minpq中，否则将*x*插入到maxpq中。
- 调整两个堆的整数个数：若minpq中元素个数较少，取出maxpq的堆顶元素插入到minpq中，若minpq比maxpq至少多2个元素，取出minpq的堆顶元素插入到maxpq中（保证minpq比maxpq至多多一个整数）。

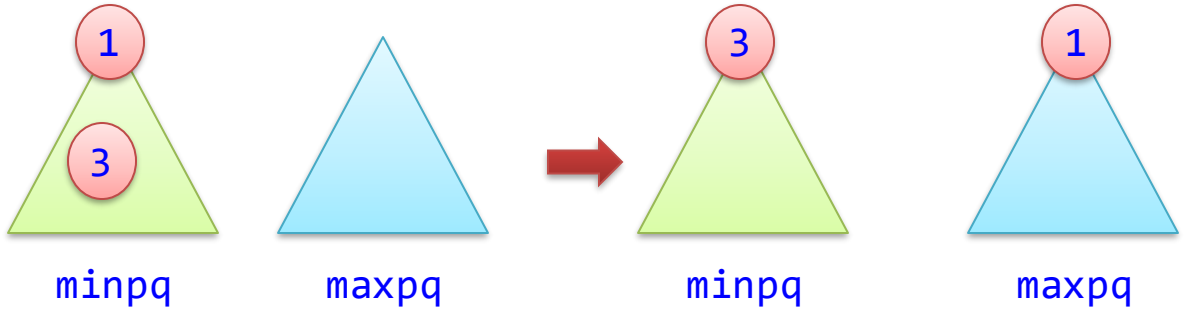
例如，输入1 3 5 4 2

1



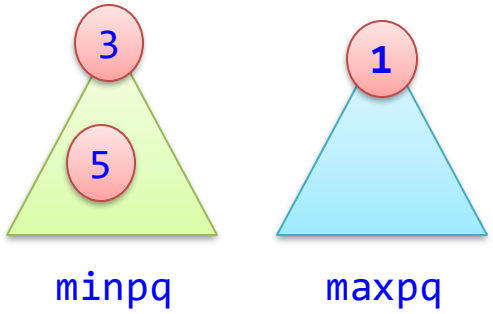
中位数为1

3



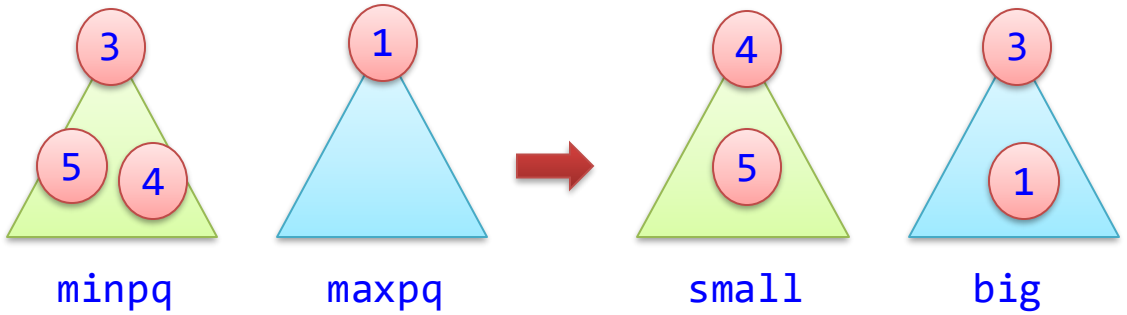
中位数为
(3+1)/2=2

5



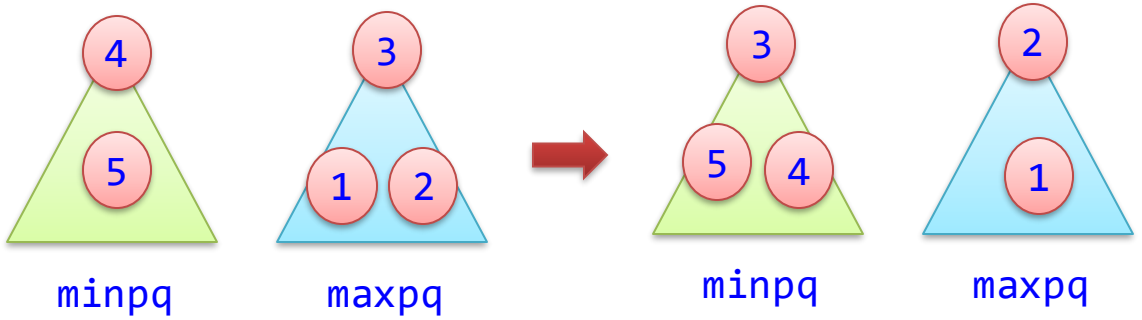
中位数为3

4



中位数为
(4+3)/2=3.5

2



中位数为3

```
priority_queue<int,vector<int>,greater<int>> minpq;  //定义一个小根堆
priority_queue<int> maxpq                            //定义一个大根堆
void addx(int x)                                     //操作1:插入x
{    if(minpq.empty())                               //若小根堆空
     {    minpq.push(x);
          return;
     }
     if(x>minpq.top())                               //若x大于小根堆堆顶元素
          minpq.push(x);                             //将x插入到小根堆中
     else
          maxpq.push(x);                             //否则将x插入到大根堆中

     while(minpq.size()<maxpq.size())                //若小根堆元素个数较少
     {    minpq.push(maxpq.top());
          maxpq.pop();                               //取出大根堆堆顶元素插入到小根堆中
     }
     while(minpq.size()>maxpq.size()+1)              //若小根堆比大根堆至少多2个元素
     {    maxpq.push(minpq.top());
          minpq.pop();                               //取出小根堆的堆顶元素插入到大根堆中
     }
}
```

```
double getmiddle()          //操作2:求中位数
{    //输入的总元素个数为奇数
     if (minpq.size()!=maxpq.size())
          return minpq.top();
     else     //输入的总元素个数为偶数
          return (minpq.top()+maxpq.top())/2.0;
}
```

```
void solve(vector<int>&v)          //求解算法
{    int i=0,x;
     while(i<v.size())
     {    if(v[i]==1)                    //操作1
          {    i++;
               x=v[i];
               addx(x);
               printf("    输入整数: %d\n",x);
          }
          else if(v[i]==2)               //操作2
          {    double mid=getmiddle();
               printf("中位数: %g\n",mid);
          }
          i++;
     }
}
```

程序验证

```
int main()
{    vector<int> v={1,2,1,6,1,3,1,2,2,1,5,2};
     solve(v);
     return 0;
}
```

